

A gentle introduction to pointer authentication

Document #: P3751R1 [[Latest](#)] [[Status](#)]
Date: 2025-11-04
Project: Programming Language C++
Audience: SG23
Reply-to: Oliver Hunt
<oliver@apple.com>
John McCall
<rjmccall@apple.com>

Contents

- [1 Introduction](#)
- [2 Revision history](#)
- [3 General overview of the hardware feature](#)
- [4 How clang models and exposes pointer authentication to C and C++](#)
- [5 General safety hazards](#)
- [6 Summary](#)
- [7 Acknowledgements](#)
- [8 Appendix](#)
- [9 References](#)

§ 1 Introduction

ARMv8.3 introduced a hardware supported control flow integrity feature called Pointer Authentication Codes, colloquially referred to as pointer auth. This paper provides a high level overview of what this feature is, how it works, and how clang implements and models the semantics of the feature.

§ 2 Revision history

- R0 (Sofia): Initial version

- R1: Explicitly address the special case fifth key present in ARMv8.3, and note that it is not relevant to normal execution.

§ 3 General overview of the hardware feature

Pointer authentication codes make use of the common observation that the high bits of a 64 bit word are unused by pointer values, and can therefore be used to carry other information. The ARMv8.3 pointer authentication extensions introduce instructions that can be used to protect these values by generating a cryptographic signature over the lower bits of and storing that signature in the high bits of the pointer. The included signature is called a Pointer Authentication Code in the extension, but in practice we refer to the combination of PAC and pointer as being either as authenticated or signed pointer.

Subsequent use of that pointer requires use of matching instructions that verify that the signature of an authenticated pointer and lead to a fault if the signature is incorrect at the time of use.

There are numerous instructions provided to support these operations, but the core semantic operations are

```
signed_pointer_t sign_pointer(raw_pointer_t ptr, key_type_t key, uint64_t
                             discriminator);
raw_pointer_t authenticate_pointer(signed_pointer_t ptr, key_type_t key,
                                  uint64_t discriminator);
```

The `key` parameter is a reference to a key stored in the processor, that is not directly accessible from code running in the process. In ARMv8.3 this is one of 4 keys, the technical details of these keys are not relevant to the semantic behavior of the operations. ARMv8.3 does include one additional special purpose key, however it is not used, nor usable in standard authentication or execution so is not relevant to this discussion.

The `discriminator` parameter is a value that is used to permit contextual separation of signed values, such that a value signed for use in one context cannot be successfully authenticated in a different context. For people with experience cryptographic contexts this is what you would usually call a “salt”.

In order to support more complex discriminators there is an additional operation provided:

```
uint64_t blend_discriminator(uint64_t value1, uint64_t value2);
```

This operation is used to provide a mechanism to combine constant and runtime values together to create a value that is usable as a discriminator for the sign and authenticate operations.

These three operations can be combined in different ways to achieve different safety and performance characteristics, depending on what is desired for any given use case. In clang we call the exact configuration the *authentication schema*.

§ 4 How clang models and exposes pointer authentication to C and C++

This section gives an overview of how pointer authentication is supported and modeled in clang, and how it is exposed to developers. These behaviors are specific to how clang has chosen to model this feature, and is not inherent to the behavior of the ISA itself.

§ 4.1 Authentication schemas

While clang provides intrinsics to directly use the instructions introduced to support pointer authentication, the more valuable and usable path is to have clang manage the pointer authentication operations itself. We do this by having a number of paths by which an authentication schema can be associated with a type.

There are two ways for a value to have an associated authentication schema. The first is that the platform ABI defines default schemas for different kinds of pointers, such as function pointers, member function pointers, v-table pointers, virtual function pointers within v-tables, and so on. These schemas are limited by the rules laid out in the standard for that kind of pointer; I'll talk about that in more detail below.

In addition to these implicit applications, clang also provides the `__ptrauth` qualifier which allows a developer to explicitly specify the schema that they want to use to protect a given declaration.

The basic syntax for the `__ptrauth` qualifier is

```
some_type_t __ptrauth(key, is_address_discriminated, discriminator)
```

Which is specifying a type of “some_type_t with an authentication schema using the specified key, whether the schema has address discrimination, and the specified discriminator”.

§ 4.2 Features of an authentication schema

The authentication schema describes how a value should be signed and authenticated by clang. If a pointer is signed with one schema and then authenticated with a different schema, the program will (probabilistically) crash. A schema has two components: the signing key, and an algorithm for computing the discriminator. In principle, the discrimination algorithm can be completely arbitrary. In practice, the algorithms used by clang are all very simple: a constant integer is optionally blended with the address in which the pointer is stored. The arguments to the `__ptrauth` qualifier — a signing key, a constant integer, and whether to blend in the storage address — can therefore specify any schema that uses one of these simple algorithms. Clang supports a few knobs to further tweak the schema, but they don't change any basic properties of the type, so they're not generally relevant to the discussions we've been having in the committee.

Different language features can use different implicit schemas. These choices become part of the platform ABI. The constant integer part of the schema can be chosen in several different ways. Some schemas derive the constant from the identity of a specific declaration, such as by hashing its symbol mangling; this is called “declaration diversity”. Other schemas derive the constant from the type, such as by hashing the mangling of the type; this is called “type diversity”. Other schemas simply pick a constant (often 0) that is used for all pointers of that kind. In general, using a more specific constant usually provides stronger protection but makes abstraction harder and can run afoul of language rules.

Blending the storage address into the discriminator (“address diversity”) provides an extremely strong defense against a wide variety of attacks. Without address diversity, attackers can overwrite pointers in existing objects and forge new objects with pointers copied from other, valid objects.

A pointer signed with address diversity can be copied or relocated in memory, but the pointer must be re-signed. Implicit schemas therefore can only use address diversity for pointers in objects that are never copied (such as v-tables) or when copies are allowed to involve running additional code.

§ 4.3 Impact of pointer authentication on the type system and language semantics

A lot of committee members have expressed concerns about knowing what they can or cannot do with any given pointer, how pointer authentication interacts with other features, or how invasive it is. The root of these issues is that it is not widely understood just how often types are implicitly subject to pointer authentication, and how many of those implicit schemas enforce address discrimination.

In the model exposed by clang, there are very few cases where pointers have any implicit authentication schema, and even fewer cases where the implicit schema introduces address discrimination. The overwhelming majority of these cases are in opaque/implementation defined regions like v-table pointers, compiler produced coroutine state, etc. When clang synthesizes a type that includes fields protected by different authentication schemas, it ensures that the synthesized type includes appropriate move/copy constructors, assignment operators, in order to ensure that the resulting type conforms to the semantics required by the language standard.

The language standard does constrain the options available when designing the implicit schemas to be applied as part of a platform’s ABI. As an example, function pointers and member function pointers are required to be trivially copyable, and so an implementation or platform that attempts to apply an address discriminating schema to such types implicitly would be non-conforming.

There are cases where the default authentication schema is not as strong as an author may wish it be, and in such cases they are able to use the `__ptrauth` qualifier explicitly to override the default schema and specify that a different should be used instead.

The `__ptrauth` qualifier behaves similarly in the language to some existing extended qualifiers, such as the address-space qualifiers described in the Embedded C TS [WG14 N1169]. `int *`, `int * __ptrauth(0,0,0)`, and `int * __ptrauth(0,1,0)` are all distinct types. Like `const` and `volatile`, `__ptrauth` is meaningful in the type of a gl-value, but converting that gl-value to a pr-value removes the top-level qualifier; this requires the pointer to be authenticated and re-signed. Unlike `const` and `volatile`, `__ptrauth` signifies a real representation difference in the object; as result, e.g. `int **` and `int * __ptrauth(0,0,0) *` are not similar types, and a value of the former type can only be converted to the latter as a `reinterpret_cast` (which may then cause authentication failures if the object is accessed through that pointer).

At its core, it does not matter how or where the `__ptrauth` qualifier is used, as it does not introduce any fundamentally new ideas to the language. Everything that the qualifier does is already an option for any class type.

§ 4.4 Using an authenticated pointer

When a pointer is protected by an authentication schema that clang is aware of, clang performs all the operations required to ensure that the visible semantics of the pointer are identical to that of an unprotected pointer.

This means:

- Loading or storing through a protected pointer automatically authenticates the pointer prior to the operation.
- Copying a pointer from a storage location with one authentication schema to a location with a different authentication schema with automatically re-sign the value from the old schema to the new.
- Comparing pointers will authenticate the values prior to comparisons (with some caveats).
- Calls through protected pointers authenticate the function pointers

In all of these cases clang will use special case instructions when possible, e.g. there are instructions to perform authenticated indirect branches and calls.

§ 4.5 Polymorphic objects

Most people in the committee who have learned of the existence of pointer authentication, have learned about it in the context of v-table pointers. This has come up frequently as the impact of the implicit schema used for v-table pointers includes address discrimination, and so using `memcpy`/`memmove` to move an object produces an invalid object. As a result no polymorphic objects on a platform that has adopted this schema can be copied by `memcpy`.

But the use of memcpy to copy polymorphic objects has never been sound as copy/move constructors and copy/move assignment operators are never trivial for polymorphic classes.

The rationale for this design is discussed more fully in the appendix to this document, but the brief summary is that every v-table pointer in an object is signed with an authentication schema that includes the storage address of the v-table pointer, and the type of the primary base class that introduced that specific v-table. Each slot in a v-table is authenticated with the storage address of that slot, and a discriminator derived from the original declaration that introduced that slot.

§ 4.6 Edge cases

Clang’s implementation of pointer authentication does include a number of features/behaviours to minimize the impedance mismatch with existing and historical C and C++, and to reduce the performance overhead as much as possible.

The most notable are:

- Function pointers do not get re-signed as they are cast from one type to another, because Real World C and C++ frequently convert functions pointers to/from `void*`, `intptr_t`, and similar. If such operations authenticated and signed the values as they path through the casts, it would trivially lead to a signing oracle (see the hazards section below)
- By default null pointers do not get signed or authenticated, this permits common cases like null pointer checks to not require authentication operations.
- Introducing an address discriminated value in a C struct results in a non-POD C struct, with interesting results that shall be left as an exercise for the reader.

§ 5 General safety hazards

This paper is focused on pointer authentication as modeled by clang, and as such is highly specific to clang’s implementation. But there are some general hazards to be aware for anyone working on or with pointer authentication as a security or safety feature.

§ 5.1 Signing oracles

The largest risk for pointer authentication is constructing a gadget that can be used by an attacker to correctly sign an arbitrary value of their choice. This is called a signing oracle, and is an easy mistake to introduce in “obvious” code.

For example a simple looking piece of code:

```
void *load_value();  
void *__ptrauth(...) p = nullptr;  
...
```

```

p = load_value();
...
void *load_value() {
    // interact with attacker controlled data, including any heap or stack
    // data
    // that can be manipulated
    ...
    return the_value;
}

```

will result in a correctly signed value that an attacker can control.

Avoiding such oracles requires significant care be taken to ensure that values being signed have a robust trust chain. Either by ensuring that the value has been signed consistently through every step of the processing, or by ensuring that the signed value comes from a trusted location (e.g read-only memory, program text, etc)

§ 5.2 Unintentional spills

The lack of complete control of the backend codegen means that is possible to write code in which you believe you have ensured an appropriate chain of trust, but due to idiosyncracies of codegen a value you have authenticated previously may get spilled prior to being used (or re-signed)

For example imagine something like a manual implementation of a v-table:

```

using my_object_t = struct my_object;
struct my_function_table {
    void (*func)(my_object_t*, int value);
}
struct my_object {
    my_function_table *__ptrauth(...) dispatch_table;
};
void f(my_object* obj) {
    obj->dispatch_table->func(obj, some_function());
}

```

This can easily become equivalent to

```

void f(my_object* obj) {
    auto func_ptr unprotected_tmp = obj->dispatch_table->func;
    int temp = some_function(); // spills unprotected_tmp, allowing an at-
    // tacker to overwrite unprotected_tmp
    unprotected_tmp(obj, temp)
}

```

This kind of spilling can also create a signing oracle if you write code that performs something akin to

```

void* pointer = authenticate_pointer(...)
some_function()
protected_pointer = sign(pointer, ...)

```

This particular attack can be mitigated by always assigning directly from authenticated storage to authenticated storage, in which case clang will do the correct thing. If clang's automatic support is not available, there are specific re-signing operations provided by the ARMv8.3 extension that provide combine the authentication and re-signing operation into a single instruction.

§ 6 Summary

Pointer authentication is a powerful tool to ensure control flow and data integrity, and it has been deployed at large scale in real world software. While it does introduce some interesting scenarios that the committee has not had to consider previously, the changes it causes do not have as wide an impact to developer code as some have feared, and case where authors choose to manually introduce greater protection are exposed to the language in ways that interact coherently with the rest of the type system.

§ 7 Acknowledgements

Thanks to Roger Orr, Ryan Mallon, and Jon Bauman for feedback on the initial draft of this paper.

§ 8 Appendix

The schemas clang uses to protect polymorphic objects have caused significant consternation among many people, in and outside of the committee. This section provides details on the rationale for and effect of the ABI choices we made in clang's application of pointer authentication to v-table dispatch.

§ 8.1 Rationale

It is well understood that attackers are able to use errors in software to manipulate program state in order to gain greater control of the target machine. There are a wide array of paths attackers can make use of when attacking software and many approaches are being investigated to limit the prevalence of those attacks, however the goal of mechanism like pointer authentication is not to stop such flaws, but to limit how powerful they are.

We are especially concerned about anything that is v-table like as such constructions are provide extremely powerful primitives that an attacker can use to manipulate the control flow of a program.

Recognizing that, we set out to make it as difficult as possible for an attacker to construct a state in a value may be used as a v-table on an object or in a state that is not valid.

§ 8.2 Design approach

In general when designing security features and mitigations we have to start with a set of assumptions of the amount of control an attacker already has when launching their attack on the value or data we are trying to protect. This can range from nothing at all, if we believe that what we are trying to protect is likely to be at the start of an attack chain, all the way through to arbitrary code execution in the kernel.

[*Note:*

Somewhat frustratingly given the widespread availability of documentation, tutorials, and classes, we have found that attackers persist in trying to do things that the relevant language standards clearly say are not permitted. As such we are forced to talk about what they are doing in terms of fundamental operations, any of which may require multiple actual steps by an attacker to construct and/or subsequently use. Each of these operations is called a primitive. As an example a “read primitive” allows an attacker to read some amount of memory they presumably are not intended to; an *arbitrary* read primitive means that an attacker can functionally construct a pointer to any address and read the content at that address.

— *end note*]

When designing the authentication schema we use for v-tables in C++ we assumed that the attacker had already got some degree of control of program state, but had not yet achieved full code execution. Essentially giving them:

- arbitrary read and write primitives
- arbitrary type confusion

How they got those primitives is not relevant to the mitigation, as often what happens is an attacker starts with a limited version of one primitive and can use that to construct a more powerful version of that primitive or to construct a limited version of another, and then slowly build up to arbitrary-X for any operation X that they need.

An attacker with these assumed primitives is already extremely well positioned, and is at this point looking almost exclusively for the final jump to code execution, and v-table like constructs are a common target.

To defend a v-table against an attacker with these tools it is necessary to ensure that they cannot reuse a v-table pointer from an old object, cannot use a v-table pointer from a type that does not match the static type of an object, cannot use a virtual method from an unrelated class, and cannot allow an attacker to construct or rewrite their own v-table out of the contents of others.

The design we choose achieves this to the extent possible by limiting the substitutability of v-table pointer in any given object, to that of another [sub-]object that has the same primary-base class, that was previously stored at the same location, and then tying the au-

thentication of any v-table slot to v-table in which it is defined, as well as the declaration that caused that v-table slot to exist.

Combined these strongly constrain the ability of an attacker to substitute, modify, or inter-mingle either v-table pointers or the content of v-tables.

§ 8.3 Pseudo-code approximation

In pseudo code if we were to consider something like this:

```
struct my_object {  
    virtual void func(int);  
    int some_field;  
};
```

would look something like (ignoring exact v-table layout)

```
struct my_object_vtable_ty {  
    using func_type_ptr = void(*)(my_object*, int);  
    func_type_ptr __ptrauth(somekey, 1, discriminator_for(func_type_ptr))  
        func_ptr;  
};  
my_object_vtable_ty my_object_vtable = {  
    .func_ptr = &impl  
};  
struct my_object {  
    my_object_vtable_ty *__ptrauth(somekey, 1, discriminator_for(my_objec-  
        t_vtable_ty)) vtable;  
    int some_field;  
};
```

a virtual call then becomes

```
void f(my_object* o) {  
    // o->func(x)  
    auto vtable_disc = blend_discriminator(&object->vtable,  
        discriminator_for(my_object_vtable_ty));  
    auto vtable = authenticate_pointer(object->vtable, somekey,  
        vtable_disc);  
    auto vtable_slot_disc = blend_discriminator(&vtable->func_ptr,  
        discriminator_for(my_object_vtable_ty::func_type_ptr));  
    auto virtual_func = authenticate_pointer(vtable->func_ptr, somekey,  
        vtable_slot_disc);  
    virtual_func(o, x);  
}
```

This is a high level approximation, and in practice the type derived discriminators used for v-tables are distinct from those derived in non v-table contexts, and are tied to the primary declarations of each v-table pointer or slot.

§ 8.4 Examples

We'll use the following basic hierarchy to try to demonstrate the classes of errors that the schema we have designed defends against.

```
struct A {
    virtual void f();
};

struct B {
    virtual void f();
};

struct C : A{
    virtual void f() override;
    virtual void g();
};

struct D : A{
    virtual void f() override;
    virtual void g();
};
```

Direct copying fails:

```
A a;
A b;
memcpy(&a, &b, sizeof(void*)); // Copy just the v-table
a.f(); // Pointer authentication failure due to different storage address
```

Similarly a type confusion, such as might occur following a use-after-free fails due to the type mismatch.

```
char buffer[...];
A *a = new (buffer) A;
B *b = new (buffer) B; // Simulate a use-after-free and replace the v-table pointer
a->f(); // Pointer authentication failure due to different primary base class
```

If you can induce a substitution with a different subtype of the same primary base class, the new v-table pointer *can* be loaded, and the call to the base class defined virtual method succeeds, which is still as it is possible that the erroneously dispatched method may make the assumption that *this* has the correct dynamic type. The final call fails as the call site assumes a call to `C::g` but the v-table slot it will load is for `D::g`.

```
char buffer[...];
C *c = new (buffer) C;
D *d = new (buffer) D; // Simulate a use-after-free and replace the v-table pointer
c->f(); // Pointer authentication succeeds due to shared primary base
c->g(); // Authentication fails as the resolved slot is for C::g, the found slot is
        // D::g
```

Virtual bases and additional v-tables are similarly protected, and so incorrect accesses fail or succeed in equivalent cases.

As we can see this is not a perfect defense, but it has substantially increased the complexity of using or manipulating in an attack to the extent that we no longer see them being targeted at all.

§ 8.5 Other languages

C++ is not alone in having dynamic dispatch, and we have introduced analagous protections in languages like Objective-C, Objective-C++, and Swift. In cases where developers have manually implemented different forms of dynamic dispatch they have been able to use the `__ptrauth` qualifier such that clang has been able to largely automate similar degrees of protection.

It is important that users and developers of other languages, including those that are considered “safe”, that these kinds of defenses need to be applied to any environment where they cannot guarantee all of the code running in a process is similar “safe”. Unsafe code is pernicious, and as a result can attack data that is defined and notionally only used in the safe language, and as such failing to apply similar levels of protection results in the state of the safe language being the target of an attacker that is exploiting an error in code written in a different language.

§ 9 References

[WG14 N1169] Wakker. C - Extensions to support embedded processors.
<https://open-std.org/jtc1/sc22/wg14/www/docs/n1169.pdf>